CS3530 Computer Networks
Assignment 1 Report

Pathri Vidya Praveen CS24BTECH11047
Tamminana Amrutha CS24BTECH11060
Indian Institute of Technology Hyderabad

September 2, 2026

Contents

1 Introduction

This report documents a TCP Echo Client/Server implemented in C on Linux using standard POSIX socket programming, extended across two assignment stages. Q1 replaces manual IP-address entry with hostname resolution via `getaddrinfo()`. Q2 adds two independently selectable features on top of the Q1 baseline: an adaptive application-layer compression scheme based on LZ77, and a concurrent multi-client server based on POSIX threads. Each feature is toggled by a command-line flag (`--compress`, `--multi`).

The remainder of this report is organised by feature: design, the specific problem each feature solves, its significance, and empirical results from automated benchmarks.

2 Q1: `getaddrinfo()`-based Hostname Resolution

2.1 Problem Statement

A minimal TCP client identifies its peer by a numeric IP address, typically converted from a dotted-decimal string via `inet_pton()`. This is inflexible: a real client is normally given a *hostname*, and address resolution must be delegated to the system resolver (DNS, `/etc/hosts`, NSS), which may return zero, one, or several candidate addresses of either address family. Q1 requires integrating this resolution step into the client and demonstrating, via a packet capture, that a genuine DNS exchange occurs.

2.2 Design and Implementation

The client calls `getaddrinfo()` with `hints.ai_family = AF_UNSPEC`, `ai_socktype = SOCK_STREAM`, so the resolver may return either IPv4 or IPv6 candidates. The returned `addrinfo` linked list is walked with `socket()` and `connect()` attempted on each entry in turn until one succeeds; `freeaddrinfo()` is called unconditionally once resolution is no longer needed, on both the success and failure paths. Resolver errors are reported via `gai_strerror()`, which is distinct from and not interchangeable with `errno`/`perror()`, since `getaddrinfo()` defines its own error-code namespace. The deprecated `gethostbyname()` API is not used anywhere.

2.3 Significance

Name-based addressing is the norm in deployed systems: server addresses change under DHCP, load balancing, and multi-homing, while hostnames remain stable. `getaddrinfo()` is the modern, protocol-independent successor to `gethostbyname()`/`inet_pton()`-based resolution, and correctly handling a list of candidate addresses (rather than assuming exactly one) is necessary for interoperability with dual-stack hosts.

2.4 Testing and Evidence

The client was exercised against: a local server via `localhost`; a custom hostname added to `/etc/hosts`; a synthetic dual-stack hostname where an unreachable IPv6 candidate is tried and rejected before falling back to a working IPv4 candidate, exercising the multi-address iteration logic; a non-existent hostname, verifying that `gai_strerror()` reports *"Name or service not known"* rather than crashing; and an unreachable port, verifying a clean *Connection refused* path. DNS query/response evidence was captured with `tcpdump`/Wireshark against a public hostname (since `localhost` and `/etc/hosts` entries resolve without touching the network at all).

2.5 Robustness Testing

Sequential and concurrent round-trip tests (20 sequential connections, 9 concurrent connections in batches of 3) all completed with correct echoes. A Valgrind memory-leak check was run against the client:

Table 1: Valgrind memory-safety summary for the Q1 client.

Metric	Result
Heap allocations / frees	31 / 31
Bytes leaked at exit	0
Errors reported	0

3 Q2 Feature 1: Adaptive Application-Layer LZ77 Compression

3.1 Problem Statement

The Q1 protocol transmits application data verbatim, regardless of redundancy in the payload. This wastes bandwidth on compressible content (repetitive or verbose text) and offers no mechanism to avoid the reverse problem: naively compressing already-dense data can *increase* the number of bytes transmitted. The goal is to compress messages before transmission when doing so helps, while preserving exact echo semantics.

3.2 Protocol Design

A 13-byte application-layer header precedes every payload in `--compress` mode (this header is entirely absent from Q1's own wire format, which is unaffected by this feature being available):

Table 2: Q2 Feature 1 message header layout.

Field	Size	Purpose
`compressed`	1 byte	0 = raw payload follows, 1 = LZ77 bitstream follows
`original_len`	4 bytes	Length of the original (uncompressed) message
`payload_len`	4 bytes	Number of bytes following the header on the wire
`compressed_bits`	4 bytes	Exact bit-length of the compressed bitstream

Multi-byte fields are serialised in network byte order via explicit `htonl()`/`ntohl()` calls into a fixed byte buffer, rather than sent as a raw `struct`, to avoid compiler-dependent padding and alignment differences between sender and receiver.

3.3 Lempel Ziv(LZ77) Compression Algorithm

The encoder performs a greedy longest-match search over a bounded sliding window (4096 bytes back, 4096-byte maximum match length), emitting either a literal byte or a (offset, length) back-reference per step. Match offsets and lengths are encoded using a universal integer prefix code (a Elias-delta-style scheme: a unary-coded length-of-length prefix, followed by the binary length field, followed by the value field itself), bit-packed directly rather than byte-aligned.

Two adaptations were required to move it into a live networking context:

1. **Fixed 256-symbol alphabet.** The original implementation computes a reduced alphabet per input string and passes it explicitly to the decoder. A live TCP receiver has no equivalent out-of-band channel for a session-specific alphabet, so literals are instead

encoded with a fixed 8-bit width covering all possible byte values – the same engineering choice made by byte-oriented compressors such as DEFLATE.

2. **Bounded window and match length.** The original implementation searches the entire processed prefix for the longest match, which is $O(n^2)$ and impractical for the tens-of-kilobyte messages used in the benchmark below. Bounding the search window is the standard LZ77/DEFLATE technique for keeping both compression time and encoded offsets bounded.

3.4 Adaptive Compression Decision

The sender always computes the LZ77-compressed representation of a message and compares its size against the original. Whichever representation is smaller is transmitted, with the header's `compressed` flag set accordingly. This avoids the failure mode of unconditional compression: on incompressible or already-dense data (e.g., random bytes), LZ77's per-literal encoding overhead can make the compressed form larger than the input, and the adaptive check ensures such messages are still sent raw.

3.5 Significance

This mirrors real protocol design choices such as HTTP's negotiated `Content-Encoding` and SSH's optional compression, where compression is applied conditionally based on payload characteristics rather than unconditionally. Demonstrating the adaptive fallback – and reporting its result honestly even when compression does not help – shows that compression benefit is a property of the data's redundancy, not solely of the algorithm. LZ77 specifically underpins DEFLATE (RFC 1951), which is the compression algorithm behind gzip-based HTTP `Content-Encoding` and SSH's `zlib` compression, making the encoder implemented here directly related to mechanisms already carrying real network traffic today.

3.6 Benchmark Methodology and Results

An automated benchmark (`benchmark_compression.py`) generates test messages of three types – *repetitive* (a short phrase repeated), *text* (original prose repeated/tiled), and *random* (`os.urandom()`) – at four sizes (50, 500, 5,000, and 50,000 bytes), and compares the bytes actually placed on the wire under Q2 Feature 1 against the Q1 baseline (which sends the raw message with no header at all).

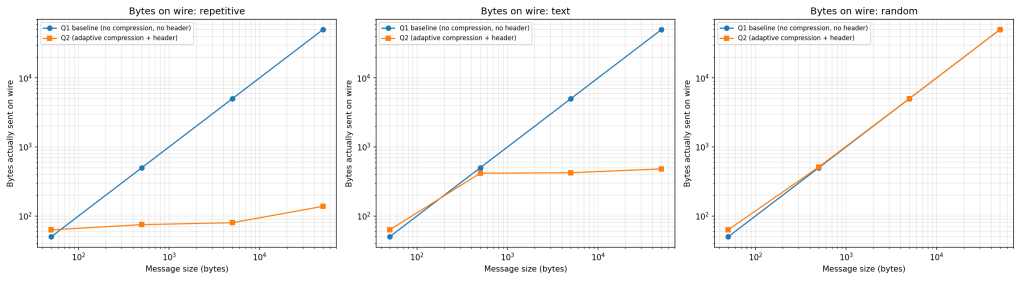

Figure 1: Bytes transmitted on the wire: Q1 baseline versus Q2 Feature 1 (adaptive compression), by message type and size.

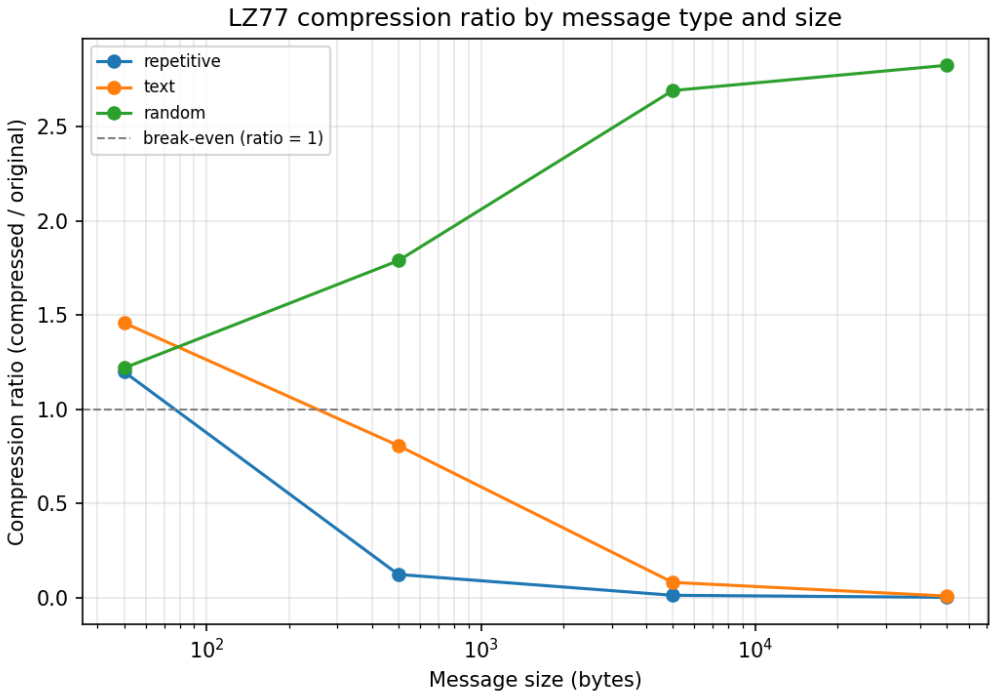

Figure 2: LZ77 compression ratio (compressed / original) by message type and size.

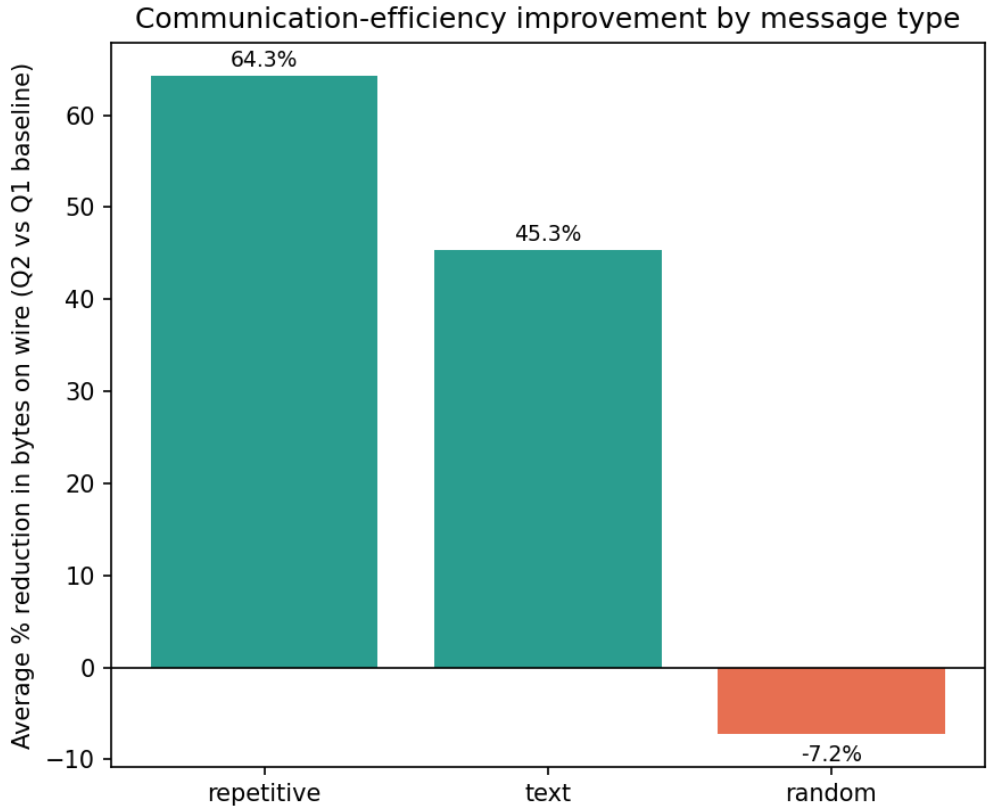

Figure 3: Average percentage reduction in bytes transmitted, by message type.

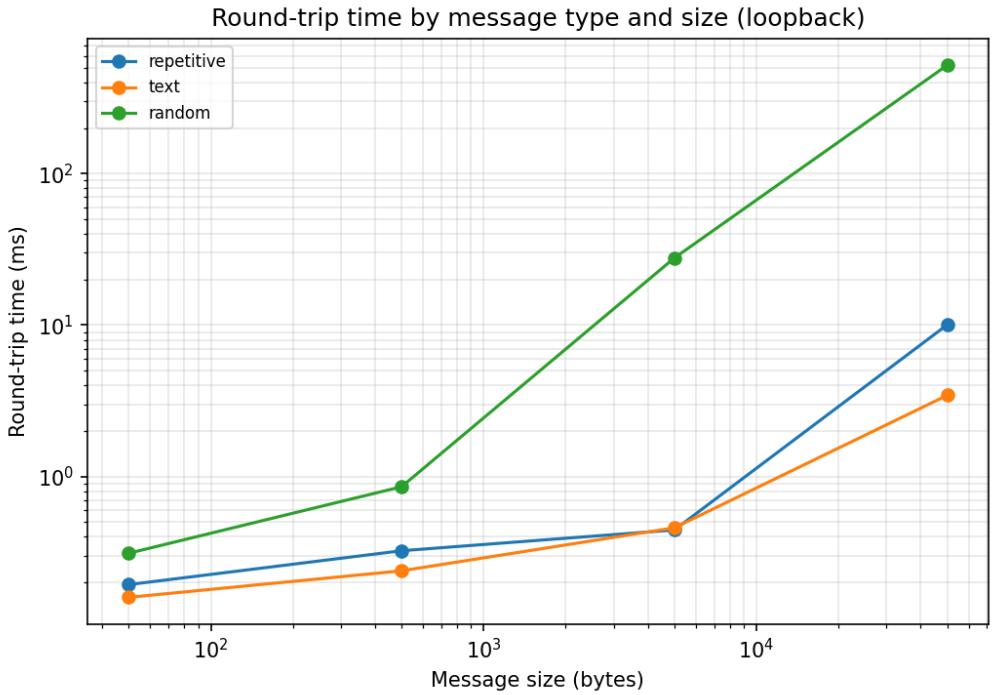

Figure 4: Round-trip time by message type and size.

Table 3: Representative benchmark results (sample run; regenerate before final submission).

Message type	Average % reduction in bytes on wire
Repetitive	+64.3%
Text	+45.3%
Random	−7.2%

The negative result for random data is expected and reported honestly: it reflects the fixed 13-byte header overhead paid even when the adaptive logic correctly declines to compress, and is smaller in magnitude than the header overhead itself would suggest precisely because compression is skipped rather than forced.

4 Q2 Feature 2: Concurrent Multi-Client TCP Echo Server

4.1 Problem Statement

The baseline server's `accept()` loop is strictly sequential: a connection is fully served (potentially many messages, in the case of Feature 1's protocol) before the next pending connection is even accepted. Under load from multiple simultaneous clients, this serialises every client's request behind every earlier one, regardless of how quickly the server could otherwise service each connection independently.

4.2 Design

A `--multi` flag on the server routes to a concurrent accept loop that spawns one `pthread` per accepted connection instead of handling it inline. Each thread invokes the same per-connection handler the sequential server already uses – the existing LZ77 handler when `--compress` is

also given, or the plain echo handler otherwise – so no protocol logic is duplicated between the sequential and concurrent server implementations. Because the choice of handler is made independently of how the connection was accepted, `--multi` and `--compress` compose freely.

Threads are created *detached* (`PTHREAD_CREATE_DETACHED`), not joined: the server's main thread has no result to collect from a finished client and must keep accepting new connections indefinitely, so joining would either block the accept loop or require tracking an unbounded, ever-growing list of thread IDs for no benefit. A detached thread's resources are reclaimed by the OS the instant it returns. No mutable state is shared between threads – each operates only on its own connection descriptor and stack-local buffers – so no explicit locking was required beyond the atomicity glibc's `stdio` already guarantees for individual `printf()` calls used in the demonstration logging.

4.3 Demonstrating Genuine Concurrency

Each thread logs its own thread ID alongside the client address it is servicing. Observing the server's output under simultaneous load shows log lines from *different* client connections interleaved with one another – for instance, client A's second message logged between client B's first and second – which is only possible if multiple connections are being actively serviced at the same time by different threads. A strictly sequential server, by construction, always logs one client's entire conversation start-to-finish before any line from the next client can appear.

4.4 Significance

This is the standard thread-per-connection concurrency model used by many production servers. Quantifying its effect – rather than merely asserting it – shows that a sequential server's total completion time for N simultaneous clients grows roughly linearly with N, since each client's full workload queues behind every earlier one, while a concurrent server's completion time remains close to flat until hardware or OS thread limits are reached.

4.5 Benchmark Methodology and Results

An automated benchmark (`benchmark_feature2.py`) compares the sequential and concurrent servers across client counts of 1, 2, 4, 8, and 16, with each of the N concurrently-launched clients sending 100 messages of 256 bytes over its own persistent connection and verifying every echo. Each configuration was repeated 3 times to reduce measurement noise. The speedup metric is defined as

$$\text{Speedup} = \frac{T_{\text{sequential}}}{T_{\text{concurrent}}} \tag{1}$$

where T is the total wall-clock time for all N clients to complete their workload.

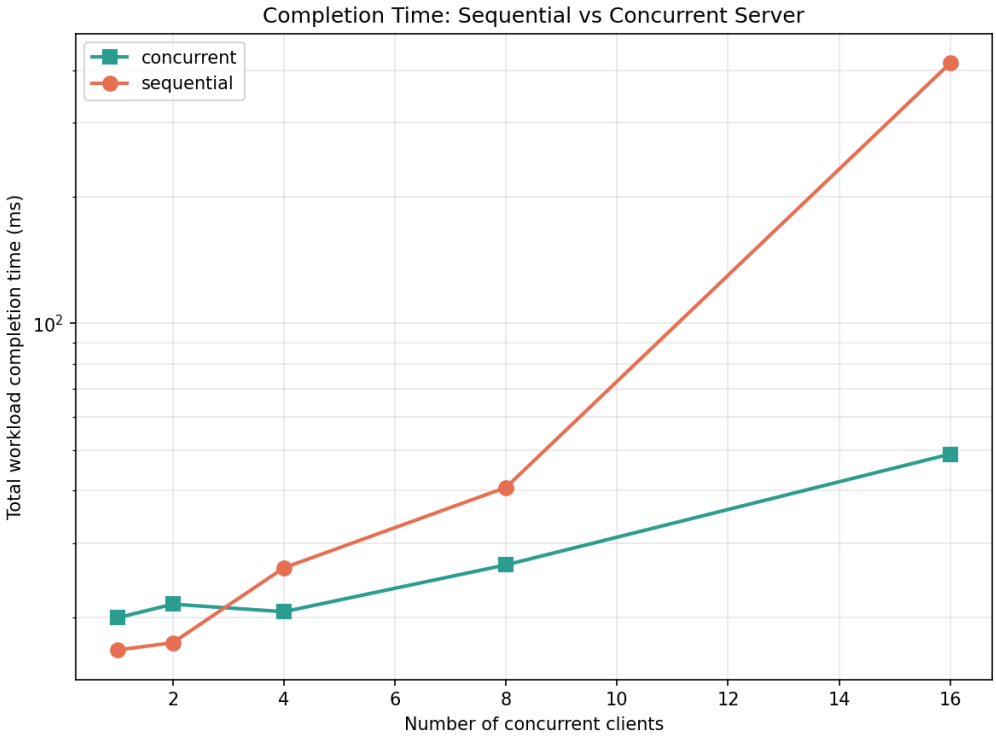

Figure 5: Total workload completion time versus number of concurrent clients (primary result).

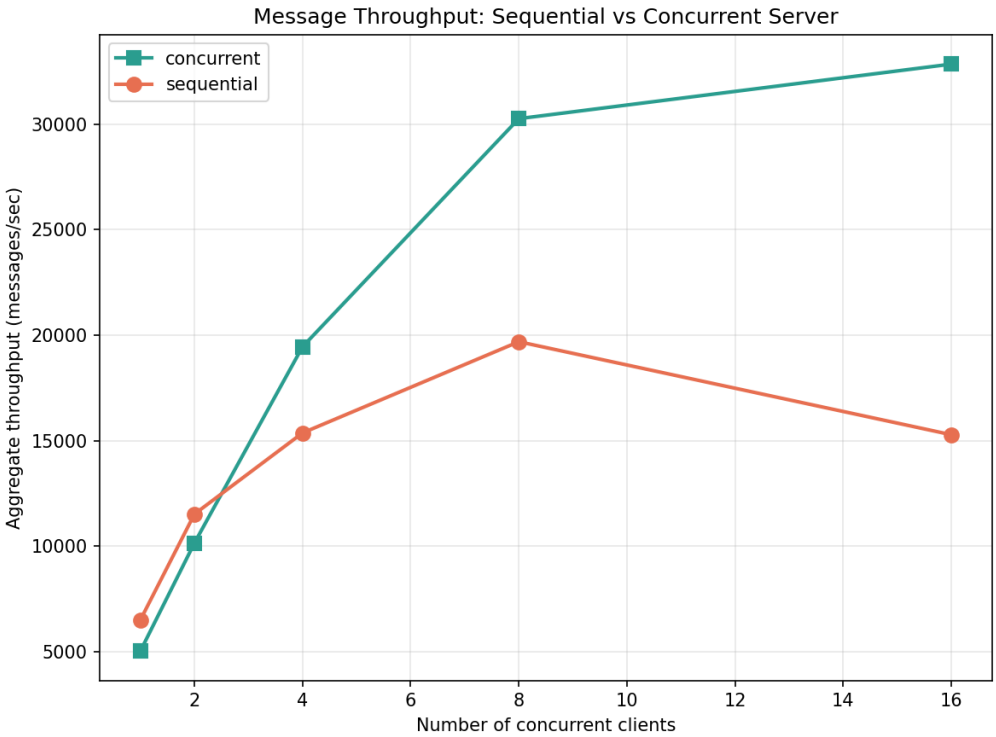

Figure 6: Aggregate message throughput versus number of concurrent clients.

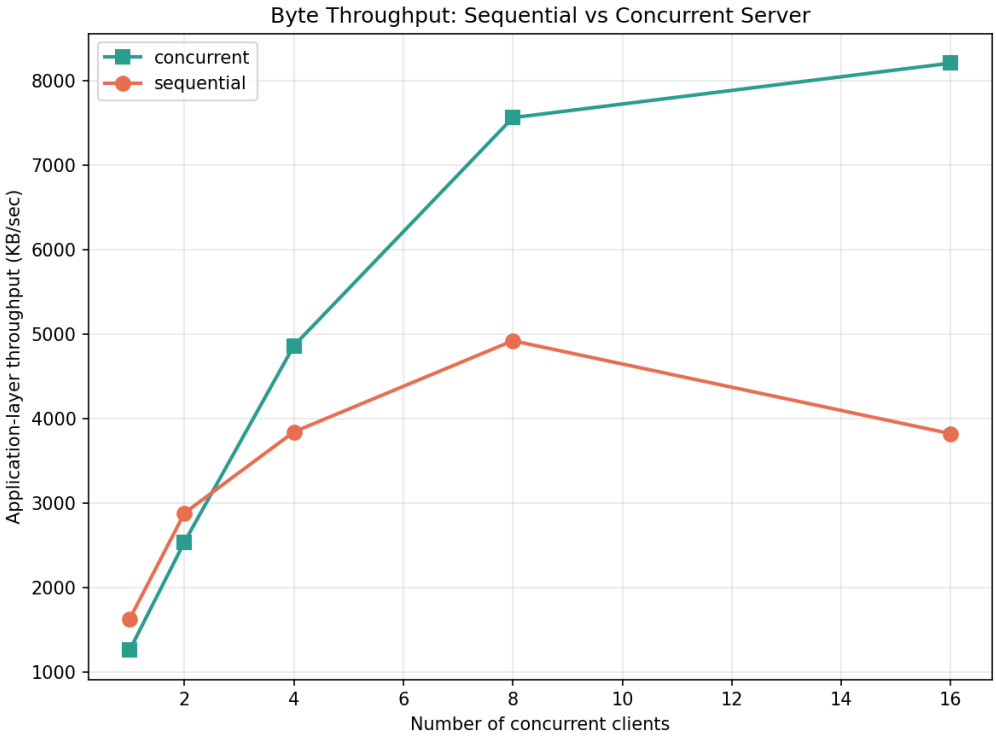

Figure 7: Application-layer byte throughput versus number of concurrent clients.

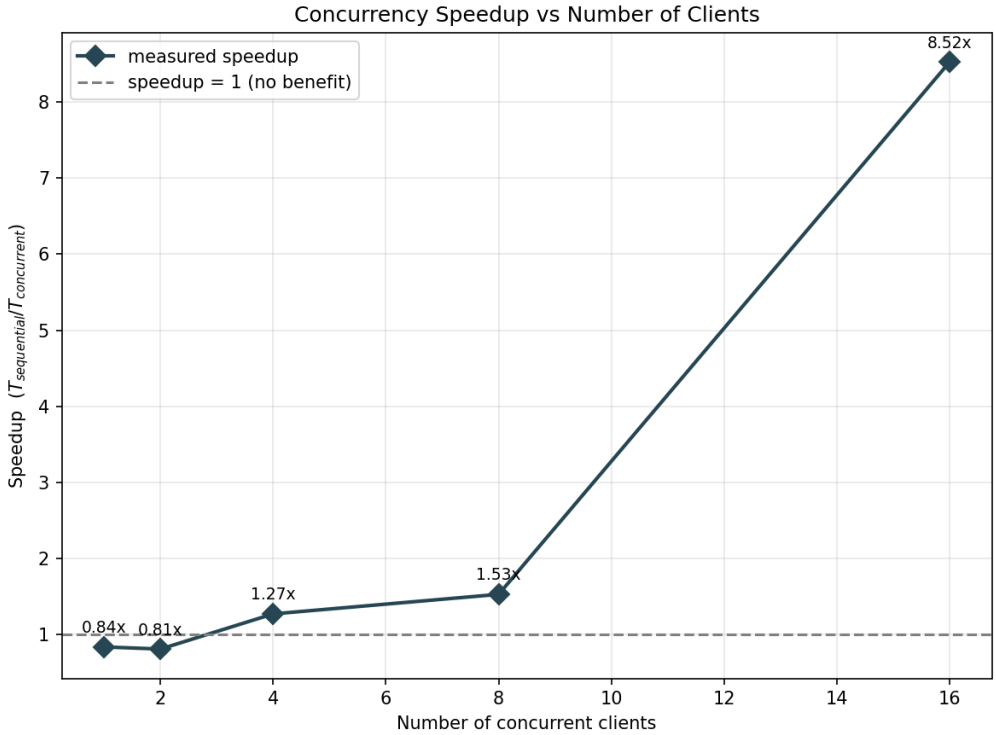

Figure 8: Measured speedup versus number of concurrent clients, with speedup = 1 shown as a reference.

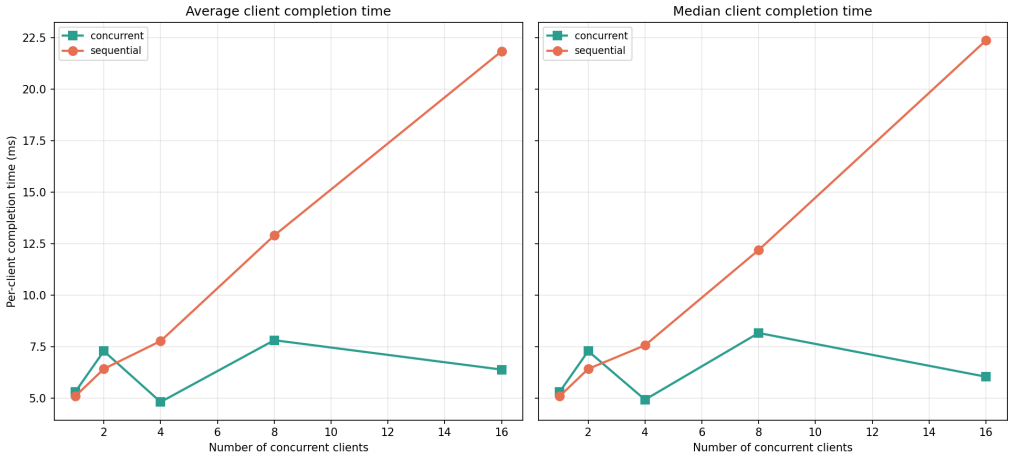

Figure 9: Average and median individual client completion time versus number of concurrent clients.

Table 4: Representative benchmark results (sample run; regenerate before final submission).

Clients	Speedup
1	∼0.84×
2	∼0.81×
4	∼1.27×
8	∼1.53×
16	∼8.52×

At the smallest client counts (1–2), the concurrent server is marginally *slower* than the sequential baseline (speedup 0.81–0.84×) – thread-creation overhead outweighs any scheduling benefit at this scale. From 4 clients onward the concurrent server overtakes the sequential one (1.27×, 1.53×), and the divergence becomes dramatic by 16 clients (8.52×), the direct, measured consequence of the sequential server's single-connection-at-a-time bottleneck. A real thread-per-connection design does carry a small fixed cost that only pays off once contention is high enough.

5 Overall Testing Methodology

Testing combined four layers: (i) functional correctness, via echo verification built into every client mode (byte-exact comparison of sent versus received data); (ii) error-path testing, covering unreachable hosts, invalid hostnames, and closed ports; (iii) memory-safety verification via Valgrind; and (iv) quantitative benchmarking via two Python automation scripts. In both benchmark scripts, Python is used exclusively for process orchestration, timing, CSV logging, and plotting – all socket programming and concurrency logic reside entirely in the C client and server binaries, which the scripts invoke as subprocesses.

6 Acknowledgement of Generative AI Usage

Generative AI (Claude, Anthropic) was used during this assignment for:

- Writing of this report and the README.md file for running the code.
- Both the Python Benchmark Files to generate plots and CSV files were generated by Claude.

- Understanding the socket programming better.
- Some assistance in debugging the code.

All the socket programming in the client.c, server.c, lz77_compression.h were entirely implemented by ourselves. All design decisions and the feature selection of LZ77 Compression and Concurrency were done by ourselves entirely by referring to the references provided at the end of the report.

7 Conclusion

The implementation satisfies Q1's hostname-resolution requirement and adds two independently useful, independently verifiable Q2 features: adaptive LZ77 compression, which reduces bytes transmitted on compressible payloads while safely declining to do so on incompressible ones, and a concurrent multi-client server, which removes the sequential bottleneck of the baseline accept loop under simultaneous client load. Both features compose with each other and with the unmodified Q1 baseline, and both are supported by automated, reproducible benchmarks whose results are reported honestly, including the cases where a feature provides little or no benefit.

References

[1] *Network Programming Basics* (lecture slides), CS3530 Computer Networks, Indian Institute of Technology Hyderabad.

[2] Course-provided reference implementations: `TCPEchoServer4.c`, `TCPEchoClient4.c`, `GetAddrInfo.c`, `TCPEchoServer4_select.c`, CS3530 Computer Networks, Indian Institute of Technology Hyderabad.

[3] Pathri Vidya Praveen, *Implementation of LZ77 and LZ78 Algorithms in Python*, EE5903 Information Theory, Coding and Inference course assignment, Indian Institute of Technology Hyderabad. `https://github.com/PathriVidyaPraveen/EE5903-Information-Theory-Coding-and-Inference`

[4] Shashank Vatedka, *EE5903 Information Theory, Coding and Inference* (course material), Indian Institute of Technology Hyderabad. `https://people.iith.ac.in/shashankvatedka/html/courses/2026/EE5903/course_details.html`

Other supporting references:

[5] Linux man-pages project, manual pages for `getaddrinfo(3)`, `gai_strerror(3)`, `socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `connect(2)`, `send(2)`, `recv(2)`, `close(2)`, `inet_ntop(3)`, `inet_pton(3)`, `pthread_create(3)`, `pthread_detach(3)`, `pthread_attr_setdetachstate(3)`, `signal(7)`. `https://man7.org/linux/man-pages/`

[6] IEEE and The Open Group, *POSIX.1-2017 (IEEE Std 1003.1-2017)*, `<netdb.h>`, `<sys/socket.h>`, `<pthread.h>` specifications.

[7] Brian "Beej Jorgensen" Hall, *Beej's Guide to Network Programming*. `https://beej.us/guide/bgnet/`

[8] James F. Kurose and Keith W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021. Chapters 2–3 (Application Layer; Socket Programming with TCP).

[9] Jacob Ziv and Abraham Lempel, "A Universal Algorithm for Sequential Data Compression," *IEEE Transactions on Information Theory*, vol. 23, no. 3, pp. 337–343, 1977.

[10] Peter Elias, "Universal Codeword Sets and Representations of the Integers," *IEEE Transactions on Information Theory*, vol. 21, no. 2, pp. 194–203, 1975.

[11] L. Peter Deutsch, *DEFLATE Compressed Data Format Specification version 1.3*, RFC 1951, IETF, 1996.

[12] Wireshark Foundation, *Wireshark User's Guide*. `https://www.wireshark.org/docs/`

[13] The Tcpdump Group, `tcpdump(1)` manual page. `https://www.tcpdump.org/manpages/tcpdump.1.html`